

Diseño e Implementación de un Lenguaje Basado en Listas y Conjuntos con Flex y Bison

Andrade, José Gabriel

Facultad de Ciencias Exactas – UNSa

Resumen:

El manejo eficiente de estructuras de datos como listas y conjuntos es una tarea central en muchas aplicaciones informáticas. Sin embargo, la mayoría de los lenguajes de programación requieren bibliotecas externas para realizar operaciones avanzadas sobre estos elementos. En este trabajo, presentaremos “**Aleph**”, un lenguaje de programación interpretado, especializado en la manipulación de conjuntos y listas, con soporte para estructuras de control y funciones definidas por el usuario. Aleph ha sido implementado utilizando Flex y Bison, con una semántica basada en “árboles de sintaxis abstracta” (AST) y un sistema de tipos dinámico. Este artículo describe la metodología utilizada en su desarrollo, los desafíos enfrentados, y los resultados obtenidos. Finalmente, se discuten las posibles aplicaciones de Aleph y las mejoras futuras en términos de optimización y extensión del lenguaje.

1. Introducción.

En este artículo se expone algunos conceptos sobre lenguajes y el diseño y la implementación de **Aleph**, un lenguaje de programación especializado en la manipulación de listas y conjuntos. El propósito principal de este trabajo es detallar la metodología empleada durante su desarrollo, así como los resultados alcanzados, proporcionando una visión completa de cómo Aleph contribuye a simplificar operaciones complejas sobre estas estructuras de datos. Además, se hablará de la importancia del desarrollo de un lenguaje enfocado en estas áreas específicas destacando el valor que aporta en términos de eficiencia y facilidad para el manejo de listas y conjuntos en diversas aplicaciones informáticas. También se hablará de las herramientas utilizadas en su creación, particularmente **Flex** y **Bison**, las cuales facilitaron la construcción del analizador léxico y sintáctico del lenguaje, permitiendo una implementación sencilla y eficiente.

1.1 Lenguajes de programación e Importancia.

Un lenguaje de programación es un sistema notacional para describir computaciones en forma legible para la maquina como también para el humano. Básicamente es un conjunto de símbolos y caracteres que permiten la comunicación usuario-maquina.

Algunas de las razones mas importantes para estudiar lenguajes de programación son:

- Incrementar la capacidad para expresar ideas.
- Mejorar el sustento para elegir lenguajes apropiados.
- Incrementar la habilidad para aprender nuevos lenguajes.

- Mejor comprensión del significado de implementación.
- Avance general en computación.

El dominio de varios lenguajes abre nuevas oportunidades de innovación, ya que los diferentes lenguajes tienen características particulares que los hacen adecuados para ciertos tipos de tareas. Por ejemplo, algunos lenguajes están diseñados para ser rápidos y eficientes mientras que otros priorizan la facilidad de uso. Esta flexibilidad y comprensión nos permite elegir las mejores herramientas para cada desafío técnico.

1.2 Tipos de lenguaje y dominios.

Los lenguajes de programación se pueden clasificar generalmente en cuatro paradigmas principales: imperativo, funcional, lógico y orientado a objetos.

1. **Lenguajes Imperativos:** se centran en cómo realizar las tareas. El programa se escribe como una secuencia de instrucciones que modifican el estado de la máquina. Ejemplos de esto son C, Java.
2. **Lenguajes orientados a objetos:** Los lenguajes Orientados a Objetos se centran en el uso de estructuras llamadas “Objetos” que modelan cosas del mundo real para resolver problemas. Ejemplos: Java, C++.
3. **Lenguajes Funcionales:** Los lenguajes funcionales se componen de funciones matemáticas puras donde su salida depende únicamente de su entrada. Ejemplos: Haskell.
4. **Lenguajes lógicos:** En este paradigma, los lenguajes se enfocan en la resolución de problemas mediante deducción lógica y reglas de inferencia, como se observa en Prolog.

Es importante reconocer que los lenguajes de programación se dividen en diferentes dominios de programación. Algunos de estos dominios incluyen:

- Aplicaciones científicas.
- Aplicaciones de negocios.
- Inteligencia artificial.
- Programación de sistemas.
- Software web.

Cada dominio responde a necesidades particulares, y comprender estos diferentes enfoques permite a los programadores crear soluciones más efectivas y adaptadas a los problemas que enfrentan.

Aleph, el lenguaje presentado en este trabajo, es principalmente **imperativo** en su naturaleza, dado que los programas en Aleph se componen de secuencias de instrucciones que manipulan estructuras de datos como listas y conjuntos. Sin embargo, Aleph también tiene características únicas que lo orientan hacia un dominio específico: el manejo eficiente de estructuras de datos. Su enfoque en listas y conjuntos lo hace particularmente adecuado para **aplicaciones científicas**.

1.3 Aleph.

Aleph es un lenguaje diseñado específicamente para la manipulación de **conjuntos** y **listas**, proporcionando operaciones nativas que facilitan su uso sin necesidad de bibliotecas adicionales. Estas estructuras de datos son ampliamente utilizadas en múltiples áreas de la computación, como la teoría de grafos, algoritmos de búsqueda y sistemas de bases de datos. Sin embargo, la mayoría de los lenguajes de programación requieren bibliotecas especializadas para trabajar con ellas, lo que puede hacer que el código sea menos intuitivo y más propenso a errores.

Los principales tres tipos de datos con los que trabajaremos en Aleph son:

Cadenas

Llamadas elementos (ELEM) dentro del lenguaje. Son simplemente cadenas de texto las cuales representan el tipo más básico de dato que puede tener Aleph. Estos se usan dentro de los conjuntos y las listas, así como también se pueden usar individualmente.

Los conjuntos (SET)

Se trata de una colección de elementos distintos y bien definidos que comparten alguna característica en común. Estos no tienen un orden definido y se pueden realizar varias operaciones con ellos. Las operaciones que ofrece Aleph son:

- **Unión:** se combinan los elementos de dos conjuntos.
- **Intersección:** elementos en común de dos conjuntos.
- **Diferencia:** elementos que están en un conjunto, pero no en otro.
- **Cardinal:** cantidad de elementos que tiene un conjunto.
- **Pertenencia:** ver si un elemento pertenece a un conjunto.
- **Inclusión:** ver si todos los elementos de un conjunto están en otro.

Las listas (LIST)

Se trata de una estructura de datos que almacena una secuencia ordenada de elementos donde los valores pueden repetirse y el orden es importante. Las operaciones sobre listas que ofrece Aleph son:

- **Push:** ingresar un elemento a la cabeza de una lista.
- **Pop:** se extrae el primer elemento de la lista.
- **Size:** cantidad de elementos que tiene una lista.
- **Pertenencia:** ver si un elemento está en una lista.
- **Concatenación:** se concatena una lista al final de otra.

Además del soporte nativo para listas y conjuntos, Aleph incorpora estructuras de control como **if**, **while** y **for each**, lo que permite la construcción de programas más expresivos. También incluye **funciones definidas por el usuario**, con soporte para paso de parámetros y validación de tipos, lo que facilita la reutilización de código y el desarrollo estructurado.

2. Metodología

La implementación de **Aleph** se realizó utilizando **Flex y Bison** para el análisis léxico y sintáctico, respectivamente las cuales son herramientas diseñadas para escritores de compiladores e intérpretes. Además, se desarrolló un evaluador semántico en **C**, basado en árboles de sintaxis abstracta (**AST**) y una tabla de símbolos para la gestión de variables y funciones. Los archivos **.c** del evaluador y los generados por **Flex**, se compilaron utilizando el compilador **MinGW**.

2.4 Implementación

Una implementación de lenguaje de programación es un sistema para ejecutar programas de ordenador. Los métodos de implementación más utilizados son:

- **Compiladores:** Un compilador es una herramienta que toma el código fuente escrito en un lenguaje de programación y lo traduce a un lenguaje de nivel inferior, generalmente código máquina, que es entendido y ejecutado directamente por el hardware de la computadora.
- **Intérpretes:** Los intérpretes ejecutan el código fuente línea por línea, traduciendo y ejecutando cada instrucción en tiempo real. Esto permite un desarrollo más rápido, ya que no es necesario compilar antes de ejecutar. Sin embargo, la ejecución puede ser más lenta en comparación con los programas compilados, ya que se necesita tiempo para traducir y ejecutar cada línea.
- **Implementación Híbrida:** Algunos sistemas de implementación de lenguajes surgen de una mezcla entre los compiladores e intérpretes puros; traducen programas de lenguaje de alto nivel a un lenguaje intermediario diseñado para permitir la fácil interpretación. Este método es más rápido que la interpretación pura porque las declaraciones de lenguaje fuente se decodifican sólo una vez. Estas implementaciones se denominan sistemas de implementación híbrida.

El enfoque de implementación que adopta **Aleph**, para la ejecución de programas y sentencias es **interpretado**, ya que su código fuente no se compila a un archivo binario ejecutable, sino que es procesado en tiempo de ejecución por un **intérprete** implementado en **C**.

2.1 Análisis Léxico – Flex

La primera fase en la compilación o interpretación de un lenguaje de programación es el **análisis léxico**, el cual, se trata del proceso de tomar una cadena de caracteres, o, más simplemente, texto, y convertirlo en grupos significativos llamados tokens.

Para el desarrollo de **Aleph**, se utilizó **Flex** como herramienta para la generación del analizador léxico, definiendo reglas para identificar los elementos sintácticos del lenguaje, como palabras clave (IF, WHILE, FOR_EACH, LET), operadores (UNION, INTER, DIFF), identificadores y elementos literales.

Flex es una herramienta que se utiliza para generar analizadores léxicos en C. Su función principal es leer un archivo de entrada con reglas de expresiones regulares y generar código en C que puede reconocer patrones en un texto de entrada, clasificarlos en tokens y pasarlos a un analizador sintáctico como **Bison**. El proceso de uso de Flex comienza con la escritura de un **archivo.l**, donde se especifican las reglas léxicas. Luego de esto, Flex genera un archivo **lex.yy.c** con el código C del analizador léxico, que se compila junto con un programa principal para producir un ejecutable. Durante la ejecución, el analizador procesa la entrada según las reglas definidas, identificando elementos como números, palabras clave o identificadores.

Las reglas léxicas de Flex son de la forma:

<patrón> {<acción>} donde “patrón” es una expresión regular y la “acción” se define en C.

Ejemplo de regla léxica para identificar identificadores y elementos:

```
$[a-zA-Z]+ { yyval.s = lookup(yytext); return ID; }  
[a-zA-Z0-9]+ { yyval.c = strdup(yytext); return ELEM; }
```

Se empleó una estrategia para diferenciar variables (ID) de valores literales (ELEM), permitiendo un procesamiento eficiente en la fase sintáctica.

Implementación

Creamos un archivo con extensión **.l**, el cual nos indica que es un archivo de Flex.

El archivo de Flex cuenta con las siguientes secciones:

1. **Sección de declaraciones (entre %{ y %})**: Se incluyen bibliotecas necesarias (stdio.h, string.h), así como los archivos de cabecera del analizador (tpt.tab.h, aleph.h). También pueden definirse variables globales.
2. **Sección de reglas (después de %%)**: Se especifican expresiones regulares y las acciones que deben ejecutarse al reconocer cada patrón.
3. **Sección de código auxiliar (%% después de las reglas)**: Puede contener funciones adicionales o la función main() que inicializa el análisis léxico con yylex(). Esta sección es opcional.

A continuación, se muestra el archivo **.l** utilizado en Aleph:

```

1  %{
2  #include <stdio.h>
3  #include <string.h>
4  #include "tpt.tab.h"
5  #include "aleph.h"
6  %}
7  %%
8  " , " |
9  " ; " |
10 "{" |
11 "}" |
12 "[" |
13 "]" |
14 "<" |
15 ">" |
16 "=" { return yytext[0]; }
17
18 "&&" { return AND; }
19 "||" { return OR; }
20 "!" { return NOT; }
21 "true" { return V; }
22 "false" { return F; }
23
24 "==" { yylval.cmp = 4; return CMP; }
25 "!=" { yylval.cmp = 5; return CMP; }
26 "<" { yylval.cmp = 6; return CMP; }
27 ">" { yylval.cmp = 7; return CMP; }
28 "<=" { yylval.cmp = 8; return CMP; }
29 ">=" { yylval.cmp = 9; return CMP; }
31 "if" { return IF; }
32 "else" { return ELSE; }
33 "while" { return WHILE; }
34 "for" { return FOR_EACH; }
35
36 "+" { return UNION_CONCAT; }
37 "^" { return INTER; }
38 "-" { return DIFF; }
39 "cardinal" { return CARDINAL; }
40 "size1" { return SIZE1; }
41 "pop" { return POP; }
42 "push" { return PUSH; }
43 "in" { return IN; }
44
45 "let" { return LET; }
46 "define" { return DEF; }
47 "return" { return RETURN; }
48
49 "print" { return PRINT; }
50
51 "[a-zA-Z]+" { yylval.s = lookup(yytext); return ID; }
52 "[a-zA-Z0-9]+" { yylval.c = strdup(yytext); return ELEM; }
53 "fn_"[a-zA-Z0-9]+" { yylval.s = lookup(yytext); return FUNC; }
54
55
56 "\n" { return EOL; }
57 [ \t ] {}
58 %%

```

En el archivo observamos la especificación de los operadores (+,-,=), así como también definimos delimitadores ({ }, ;) y palabras clave para cada una de las estructuras de control como (for, while, if, let).

2.2 Análisis Sintáctico - Bison

Después del análisis léxico, se realiza el **análisis sintáctico**. En esta etapa se verifica si los tokens forman estructuras sintácticamente correctas según las reglas de gramática del lenguaje. El resultado es un **árbol de sintaxis abstracta (AST)**, que representa la estructura jerárquica del código fuente.

El análisis sintáctico fue desarrollado con **Bison**, definiendo una gramática que soporta expresiones, operaciones sobre conjuntos y listas, estructuras de control y funciones.

Bison es una herramienta de generación de analizadores sintácticos utilizada en el desarrollo de compiladores e intérpretes. Su función principal es procesar una secuencia de tokens generada por un analizador léxico, como el producido por Flex, y organizarlo de acuerdo con una gramática formal. De esta manera, verifica que la estructura de la entrada sea válida y genera una representación estructurada de la misma, comúnmente en forma de un árbol de sintaxis abstracta (AST).

Para definir la sintaxis del lenguaje, Bison utiliza **gramáticas libres de contexto (GLC)** expresadas en la notación **Backus-Naur Form (BNF)**. A partir de estas reglas, Bison genera un analizador en C capaz de manejar expresiones, estructuras de control y otros elementos del lenguaje de manera jerárquica.

Las reglas sintácticas de Bison son de la forma:

<nombre> : <producción> {<acción>} , donde nombre es un no terminal, producción son las formas en las que se puede construir esa regla y la acción la cual se define en C.

Ejemplo de reglas para operaciones entre listas y conjuntos:

```
exp:  exp INTER exp      { $$ = newast(INTER, $1, $3); }
    |  exp UNION exp     { $$ = newast(UNION, $1, $3); }
    |  POP exp           { $$ = newast(POP, NULL, $2); }
```

Implementación

Continuamos con la creación del archivo con extensión **.y**, lo que indica que es un archivo de **Bison**. El archivo Bison cuenta con las siguientes secciones:

1. **Declaraciones en C** (entre **%{ y %}**): Aquí se incluyen bibliotecas, definiciones de variables globales y estructuras necesarias para el analizador.
2. **Declaraciones de Bison**: Se definen tokens, tipos de datos, asociatividad y precedencia de operadores.
3. **Reglas gramaticales (después de %%)**: Se especifican las reglas de la gramática, donde cada regla se asocia a una acción en C que construye el AST o evalúa expresiones.
4. **Código C adicional (después de %%)**: Puede contener funciones auxiliares como `yerror()` o `main()`, necesarias para ejecutar el analizador sintáctico.

A continuación, se muestra el código del archivo Bison utilizado en la implementación de Aleph:

```
1  %{
2  #include <stdio.h>
3  #include "aleph.h"
4
5  DataType dt = NULL;
6
7  extern int yylex();
8  int yyerrork;
9  void yyerror(char *);
10 %}
11
12 %union{
13     struct ast* a;
14     struct symbol *s;
15     struct symlist *sl;
16     char* c;
17     int cmp;
18 }
19
20 %type <a> exp
21 %type <a> literal_conj
22 %type <a> literal_list
23 %type <a> list_elem
24 %type <sl> parametros
25 %type <a> stmts
26 %type <a> aleph
27 %type <a> control
28 %type <a> cond
```

Primero definimos los tipos de datos que se utilizarán y establecemos qué tipo de datos corresponde a cada token. Luego, definimos la precedencia y asociatividad de los operadores, lo que evita errores de ambigüedad al procesar expresiones.

```

52 aleph: /*vacío*/          { $$ = NULL; printf("\naleph> "); }
53 | aleph error EOL        { yyerrork; printf("\naleph> "); }
54 | aleph stmts EOL        { dt = eval($2); freeast($2); printf("\naleph> "); }
55 | aleph DEF FUNC '('parametros')' '{'stmts'}' EOL    { newfn($3, $5, $8); printf("\naleph> "); }
56 | aleph DEF FUNC '('')' '{'stmts }' EOL            { newfn($3, NULL, $7); printf("\naleph> "); }
57 | aleph LET '('parametros')' '=' '('list_elem')' EOL { newasgnmult($4, $8); printf("\naleph> "); }
58 ;
59
60 stmts: stmts ';'          { $$ = newast(';', $1, NULL); }
61 | stmts ';' exp          { $$ = newast(';', $1, $3); }
62 | stmts ';' control      { $$ = newast(';', $1, $3); }
63 | control                { $$ = $1; }
64 | exp                    { $$ = $1; }
65 ;
66 ;
67
68 control: IF '(' cond ')' '{' stmts '}' { $$ = newif(IF, $3, $6, NULL); }
69 | IF '(' cond ')' '{' stmts '}' ELSE '{' stmts '}' { $$ = newif(IF, $3, $6, $10); }
70 | WHILE '(' cond ')' '{' stmts '}' { $$ = newast(WHILE, $3, $6); }
71 | FOR_EACH ID IN exp '{' stmts '}' { $$ = newfor(FOR_EACH, newasgn($2, NULL), $4, $6); }
72 ;

```

La estructura de nuestro código Bison se divide en 9 variables sintácticas principales, pero nos centraremos en explicar solo las 4 más importantes. Primero tenemos a *aleph*, la cual es el punto de entrada del programa y permite definir funciones, declaraciones y sentencias. La variable *stmts* representa una lista de sentencias, permitiendo ejecutar múltiples expresiones separadas por ;. De la variable *control*, derivan todas las reglas que tienen que ver con las estructuras de control de Aleph.

```

74 exp: exp UNION_CONCAT exp { $$ = newast(UNION_CONCAT, $1, $3); }
75 | exp INTER exp          { $$ = newast(INTER, $1, $3); }
76 | exp DIFF exp           { $$ = newast(DIFF, $1, $3); }
77 | POP exp                { $$ = newast(POP, NULL, $2); }
78 | exp PUSH exp           { $$ = newast(PUSH, $1, $3); }
79 | CARDINAL exp           { $$ = newast(CARDINAL, NULL, $2); }
80 | SIZEL exp              { $$ = newast(SIZEL, NULL, $2); }
81 | literal_conj           { $$ = $1; }
82 | literal_list           { $$ = $1; }
83 | '(' exp ')'            { $$ = $2; }
84 | ID '=' exp             { $$ = newasgn($1,$3); }
85 | ELEM                   { $$ = newelem(STR, $1); }
86 | ID                     { $$ = newref($1); }
87 | PRINT exp              { $$ = newast(PRINT, $2, NULL); }
88 | FUNC '('list_elem')'   { $$ = newfncall($1, $3); }
89 | FUNC '('')'           { $$ = newfncall($1, NULL); }
90 | RETURN exp             { $$ = newast(RETURN, $2, NULL); }
91 ;
92
93 cond: exp IN exp          { $$ = newast(IN, $1, $3); }
94 | exp CMP exp            { $$ = newast($2, $1, $3); }
95 | '(' cond ')'          { $$ = $2; }
96 | cond AND cond         { $$ = newast(AND, $1, $3); }
97 | cond OR cond          { $$ = newast(OR, $1, $3); }
98 | NOT cond              { $$ = newast(NOT, $2, NULL); }
99 | V                     { $$ = newast(V, NULL, NULL); }
100 | F                     { $$ = newast(F, NULL, NULL); }
101 ;

```


La variable *exp* deriva en todas las otras tareas relacionadas con operaciones sobre las listas y conjuntos, asignaciones, llamada a funciones e incluso una función propia de Aleph, la cual nos permite mostrar nuestras operaciones.

Además, se definen reglas para manejar listas, conjuntos y parámetros de funciones, lo que permite una mayor flexibilidad en la sintaxis del lenguaje.

La última sección separada por `%%`, contiene 2 funciones importantes para el funcionamiento del analizador. La primera es la función `yyerror(char *s)`. Esta función se ejecuta cuando el analizador detecta un error de sintaxis en la entrada del usuario. `yyerror` recibe un mensaje de error (s), que es generado automáticamente por Bison. Luego, imprime el mensaje en `stderr` (la salida estándar de errores) para informar al usuario sobre el problema.

Luego tenemos la función `main()`, que llama a `yyparse()`, que es la función generada por Bison y se encarga de iniciar el análisis sintáctico. `yyparse()` leerá la entrada, verificará su validez según la gramática de Aleph y construirá el árbol de sintaxis abstracta (AST).

```
103 < literal_conj: '{' list_elem '}' { $$ = newast(SET, $2, NULL); }
104 |   |   |   | '{''}'           { $$ = newast(SET, NULL, NULL); }
105 ;
106
107 < literal_list: '[' list_elem ']' { $$ = newast(LIST, $2, NULL); }
108 |   |   |   | '[''']           { $$ = newast(LIST, NULL, NULL); }
109 ;
110
111 < list_elem: exp ',' list_elem { $$ = newast('L', $1, $3); }
112 |   |   |   | exp           { $$ = newast('L', $1, NULL); }
113 ;
114
115 < parametros: ID ',' parametros { $$ = newsymlist($1, $3); }
116 |   |   |   | ID           { $$ = newsymlist($1, NULL); }
117 ;
118
119 %%
120 < void yyerror(char *s) {
121 |   fprintf(stderr, "error: %s\n", s);
122 | }
123
124 < int main() {
125 |   return yyparse();
126 | }
```

Esta sección es clave para definir la estructura sintáctica de Aleph y guiar la construcción del AST. Gracias a esta implementación, el lenguaje es capaz de reconocer expresiones, estructuras de control y funciones definidas por el usuario, permitiendo su posterior evaluación en la fase semántica.

2.3 Evaluación Semántica

El **Árbol de Sintaxis Abstracta (AST)** es una estructura jerárquica que representa la sintaxis del código de manera simplificada, omitiendo elementos innecesarios como paréntesis y separadores.

Se implementó un evaluador “**eval()**” basado en estos mismos, donde cada nodo representa una operación o una estructura de control. Las expresiones se transforman en un tipo de datos unificado (**DataType**), permitiendo la manipulación de listas, conjuntos y valores individuales de una manera más sencilla.

El evaluador fue implementado en C y la estructura del **AST** se define mediante la siguiente estructura recursiva:

```
struct ast{
    int nodetype;
    struct ast *l;
    struct ast *r;
};
```

Es una estructura sencilla, donde el tipo de nodo indica la operación a realizar y los hijos representan sus operandos.

Por ejemplo, la expresión **{1,2} UNION {3,4}** se traduce en un nodo **UNION** con **{1,2}** y **{3,4}** como hijos. Como ya hemos mencionado antes, en la implementación de Aleph, el AST se genera en Bison con reglas como

```
exp: exp UNION exp      { $$ = newast(UNION, $1, $3); }
```

donde **newast()** construye un nodo con **UNION** como raíz y sus operandos como hijos. Luego, este AST es recorrido y evaluado con una función recursiva en C, que garantiza que cada nodo se procese correctamente junto con sus hijos. En la evaluación, las expresiones se transforman en objetos **DataType**, de una biblioteca que creada para facilitar su manipulación y almacenamiento.

Ejemplo de evaluación de operaciones de conjuntos:

```
case UNION_CONCAT://union y concatenacion
    if(a -> r != NULL && a -> l != NULL){
        izq = copiar(eval(a -> l));
        der = copiar(eval(a -> r));
        aux = operacion(izq, der);
    }
    else{
        aux=NULL;
    }
    break;
```

En este caso, la función **eval()** se invoca de manera recursiva para evaluar los operandos izquierdo (**a->l**) y derecho (**a->r**). Luego, se copian los valores resultantes y se pasan a la función **operacion()**, que se encarga de aplicar la unión o concatenación dependiendo de los tipos de datos.

Evaluación en el Árbol de Sintaxis Abstracta

El evaluador semántico sigue esta misma lógica para cada tipo de nodo del AST. Utiliza una estructura **switch-case** en la que cada **nodetype** representa una operación específica del lenguaje. En cada caso, se evalúan los nodos hijos y se determina cómo proceder en función de sus valores y tipos.

Chequeo de Tipos

Para garantizar la corrección semántica, antes de realizar cualquier operación se verifica que los operandos correspondan al tipo de operación a realizar. Si el chequeo de tipos falla, la operación se cancela y se muestra un mensaje de error.

Ejemplo de error en la evaluación:

Si intentamos ejecutar la operación **{1,2} + 1**, el evaluador verificará el tipo de los operandos. Primero, identificará que el operando izquierdo (a->l) es un **SET** y el derecho (a->r) es un **ELEM** (cadena que representa un número). Dado que la unión solo está definida entre conjuntos, el evaluador detectará la incompatibilidad y generará un error.

Este mecanismo de validación asegura que todas las operaciones en Aleph sean semánticamente correctas, evitando resultados inesperados y facilitando el desarrollo de programas más robustos.

Estructuras de control

Además de la estructura del AST, también se implementaron otras estructuras similares pero útiles para la manipulación de distintos datos. Como mencionamos anteriormente, Aleph cuenta con estructuras de control, las cuales cuentan con una estructura propia, lo que facilita trabajar con ellos. Las estructuras implementadas son las siguientes:

```
struct if_struct{
    int nodetype;
    struct ast *cond;
    struct ast *thn;
    struct ast *els;
};

struct for_struct{
    int nodetype;
    struct ast* var;
    struct ast* exp;
    struct ast* stmts;
};
```

Para el caso del **While**, simplemente se utilizó la estructura típica del AST.

Tabla de símbolos

Aleph cuenta con una **tabla de símbolos** que permite almacenar y gestionar identificadores, como variables y funciones. Su principal propósito es asociar cada nombre con su respectivo valor o definición, facilitando la evaluación de expresiones y la ejecución de funciones.

Una tabla de símbolos es una estructura de datos utilizada en compiladores e intérpretes para registrar información sobre identificadores a medida que se analiza el código fuente. En Aleph, esta tabla está diseñada como una **tabla hash** de tamaño fijo, lo que permite búsquedas eficientes y una rápida asignación de valores.

A continuación, presentamos la estructura de la tabla:

```
8  struct symbol{
9      char *name;
10     DataType value;
11     struct ast *func;
12     struct symlist *syms;
13 };
14
15 /* tabla de símbolos simple de tamaño fijo */
16 #define NHASH 9997
17 struct symbol symtab[NHASH];
```

Para su gestión, se implementó la función **lookup()**, que permite acceder a un identificador dentro de la tabla o agregarlo en caso de que este no se encuentre.

Las variables en Aleph son **globales** y se diferencian de los elementos literales porque comienzan con el símbolo \$. Esta distinción permite diferenciar entre referencias a variables y valores directos dentro del lenguaje. Cuando se accede a una variable, el intérprete busca su valor en la tabla de símbolos y lo retorna.

Asignaciones simples y múltiples

Las asignaciones en Aleph son manejadas por la siguiente estructura:

```
34
35 struct symasgn{
36     int nodetype;      /* tipo = */
37     struct symbol *s;
38     struct ast *v;     /* valor */
39 };
40
```

Al evaluar una asignación, el evaluador almacena el valor en la tabla de símbolos, lo que garantiza que las variables mantengan su valor durante la ejecución del programa.

Además de la asignación simple, Aleph soporta **asignaciones múltiples**, lo que permite asignar valores a varias variables en una sola línea, mejorando la legibilidad y eficiencia del código.

Internamente, la asignación múltiple se representa con la siguiente estructura de datos:

```
65
66 struct asgnmult{
67     int nodetype;
68     struct symlist *vars;
69     struct ast *vals;
70 };
71
```

La estructura cuenta con una lista de símbolos, las cuales son del tipo **symbol** visto anteriormente que representan las variables a asignar y con un **AST** vals, con los valores correspondientes.

Durante la evaluación, el intérprete recorre ambas estructuras en paralelo, asignando los valores a las variables en la tabla de símbolos. Si el número de variables no coincide con el de valores, se genera un error, pero se asignan las variables que se pueden.

Funciones

Las funciones también se almacenan en la tabla de símbolos, utilizando los campos **func** y **syms** de la estructura **symbol** que representan el cuerpo de la función y la cantidad de parámetros de la misma.

Cuando se define una función, se llama **newfn()** el cual asigna las variables y parámetros al símbolo de la función declarada.

Una vez definida la función, esta podrá ser llamada por el usuario haciendo uso de la función **newfncall()** y la estructura **fncall**:

```
71
72  struct fncall{
73      int nodetype;
74      struct symbol *f;
75      struct ast *param;
76  };
77
```

Dentro el evaluador, tomamos los parámetros pasados de esta estructura, asignándolos a los parámetros de la definición de la función (esto es para que cuando se evalué el cuerpo de la función, los valores utilizados sean los pasados en la llamada a la misma) y evaluamos el cuerpo de la función.

3. Resultados

La implementación de **Aleph** resultó en un lenguaje funcional capaz de:

- Definir y manipular listas y conjuntos.
- Evaluar expresiones y ejecutar estructuras de control.
- Permitir la definición de funciones con parámetros.

Ahora procederemos a mostrar los resultados de las distintas funciones del lenguaje:

3.1 Compilación de archivos

Para poner en marcha el lenguaje, primero debemos posicionarnos dentro de la carpeta que contiene los archivos C, los archivos de Flex y los archivos de Bison. Cuando estemos en ella, primero vamos a procesar el código de Flex, lo que genera el analizador léxico **lex.yy.c**. Seguido, procesamos el código de Bison, generando así el analizador sintáctico **tpt.tab.c**.

Finalmente compilamos el código C del evaluador, así como también los generados por Flex y Bison. Hecho esto, se nos generara un archivo ejecutable **a.exe**. Al ejecutarlo podremos escribir el código del lenguaje.

```
PS C:\TPT\Final> flex tpt.l
PS C:\TPT\Final> bison -d tpt.y
PS C:\TPT\Final> gcc lex.yy.c tpt.tab.c TADdataType.c aleph.c -lfl
PS C:\TPT\Final> .\a.exe
```

3.2 Listas y Conjuntos

Dentro del lenguaje podemos escribir listas y conjuntos literales, pudiendo realizar operaciones con ellos. Usamos la función **Print()** para poder observar los resultados de las operaciones:

```
aleph> print( {1,2} + {1,4} )
{1,2,4}

aleph> print( [1,2] + [1,4] )
[1,2,1,4]

aleph> print( pop [100,200] )
100

aleph> print( 100 push [] )
[100]

aleph> print( cardinal {1,2,3} )
3
```

El operador “+” se utiliza tanto para la unión de conjuntos como para la concatenación de listas. Se le conoce como operador **sobrecargado**.

```
aleph> print( size1 [1,2,{1,[2]}] )
3

aleph> print( {1,2,[100]} ^ {[100],1,4} )
{[100],1}

aleph> print( {1,2,[100]} - {[100],1,4} )
{2}
```

3.3 Asignación y Asignación múltiple

Como ya mencionamos, usamos el símbolo \$ para distinguir las variables de los elementos.

```

aleph> $varA = {1,2,1,{1},{1}}

aleph> print($varA)
{1,2,{1}}

aleph> $varB = {20,2,{1},[1,2]}

aleph> print($varB)
{20,2,{1},[1,2]}

aleph> $resultado = $varA ^ $varB

aleph> print($resultado)
{2,{1}}

```

Para la asignación múltiple utilizamos la palabra reservada **Let**. Seguido colocamos las variables en el orden que se quiere asignar luego del = .

En caso de que los valores a asignar sean mas que las variables disponibles, se asignara hasta la ultima variable disponible y se perderán los valores restantes y en el caso contrario que sean mas variables que valores a asignar, se mostrara un mensaje de **error** y a las variables sobrantes se les asignara el **elemento nulo**.

```

aleph> let ($A, $B, $C) = ({1}, [1], {2}+{1})

aleph> print($A); print($B); print($C)
{1}
[1]
{2,1}

aleph> let ($A, $B) = ([2], {1,2}, [100, 500])

aleph> print($A); print($B)
[2]
{1,2}

aleph> let ($varA, $varB, $varC) = ({500})
error: algunas variables no se asignaron.

aleph> print($varA); print($varB); print($varC)
{500}

elemento nulo

elemento nulo

```

3.4 Estructuras de Control

Ahora presentaremos ejemplos del funcionamiento de las estructuras de control.

Sentencias **if**: si 1 pertenece al conjunto y si un conjunto está incluido en otro.

```

aleph> if( 1 in {320, 411, 1, [2]} ) { print(pertenece) } else { print(noPertenece) }
pertenece

aleph> if( {200} < {1, {20}, 90} ) { print(incluido) } else { print(noIncluido) }
noIncluido

aleph> if( 1 in {1,2} && {20} < {10,20} ) { print(verdadero) } else { print(falso) }
verdadero

aleph>

```

Sentencias **while**: mientras una lista tenga elementos, mostrarlos y sacarlos.

```

aleph> $var = [1,2,3,{1,2}]

aleph> while( (size $var) > 0 ){ print(pop $var) }
1
2
3
{1,2}

aleph> print($var)
[]

```

Sentencias **for each**: para cada elemento de un conjunto de números, contar los menores que 10.

```

aleph> $cont = 0

aleph> for $var in {32,1,0,12,7,100} { if( $var < 10 ){ $cont = $cont + 1 } }

aleph> print($cont)
3

aleph>

```

3.5 Definición de Funciones

Definimos funciones con la palabra reservada **define**. Mostraremos el siguiente ejemplo: dado una lista, sacar todos sus elementos y ponerlos en un conjunto.

```

define fn_listToSet($list){
    $conj = {};
    while((size $list) > 0){
        conj = $conj + {pop $list};
    };
    return $conj;
}

```



```
Funcion "fn_listToSet" definida.  
  
aleph> $var = fn_listToSet([1,2,3,{4}])  
  
aleph> print($var)  
{1,2,3,{4}}  
  
aleph>
```

4. Conclusiones

El desarrollo de Aleph demuestra cómo un lenguaje de programación especializado puede optimizar la manipulación de listas y conjuntos, proporcionando una sintaxis clara y eficiente para operaciones que en otros lenguajes requieren bibliotecas externas. Al ser un lenguaje interpretado con un sistema de tipos dinámico, Aleph ofrece flexibilidad y facilidad de uso, permitiendo a los programadores trabajar con estructuras de datos de manera intuitiva.

Dado que Aleph está orientado al desarrollo aplicaciones científicas, su diseño prioriza la expresividad y la simplicidad en la manipulación de datos estructurados. Sin embargo, este se ve limitado en ciertos aspectos y existen oportunidades de mejora.

4.1 Limitaciones

La principal limitación de Aleph viene dada por la limitada cantidad de tipos de datos. El tipo de dato atómico ELEM no distingue entre cadenas de texto y números, por lo que debe ser tratado internamente dependiendo del contexto en el que se use aumentando la complejidad en algunas operaciones. Las funcionalidades que ofrece distinguiendo de números o cadenas son bastante simples. Por esta razón no es posible realizar operaciones mas complejas como divisiones, potencias o incluso restas.

La falta de funciones nativas es otro problema. Si bien Aleph permite la creación de funciones definidas por el usuario, el lenguaje carece de muchas funciones propias para operaciones avanzadas. Esto obliga a los programadores a implementar manualmente muchas funcionalidades que en otros lenguajes estarían disponibles de forma predeterminada. Además, dado que Aleph no se puede enlazar con bibliotecas externas, su capacidad de expansión y reutilización de código existente se ve limitada.

El uso exclusivo de variables globales impide la existencia de ámbitos locales dentro de las funciones. Esto puede generar efectos secundarios o valores erróneos, así como también el aumento de la complejidad para escribir funciones y el mantenimiento del código en programas muy grandes.

La **falta de procedimientos** es otra limitación de Aleph. Al solo permitir funciones, Aleph limita el paso de parámetros para que este solo sea por valor. Los parámetros no pueden ser pasador por referencia, por lo que los procedimientos no están contemplados en Aleph. Además de esto, tampoco es posible definir funciones recursivas, limitando así la implementación de algoritmos basados en esta técnica.

4.2 Posibles Mejoras

Para abordar las limitaciones mencionadas, Aleph podría beneficiarse de diversas mejoras y extensiones en futuras versiones.

La ampliación del sistema de tipos, introduciendo tipos propios para cadenas y enteros permitiría realizar operaciones más avanzadas, como divisiones, potencias y restas. Además, la inclusión de nuevos tipos de datos, como booleanos y estructuras de datos personalizadas, mejoraría la expresividad del lenguaje.

Agregar **soporte para procedimientos y paso por referencia** permitiría una manipulación más eficiente de estructuras de datos grandes, además de facilitar la implementación de algoritmos complejos. Asimismo, la inclusión de **variables locales** dentro de funciones reduciría los efectos secundarios y contribuiría al desarrollo de programas modulares, mejorando la organización y mantenibilidad del código.

Incorporar soporte para **funciones recursivas** mejoraría la expresividad y la capacidad de resolución de problemas del lenguaje.

Otra posible mejora sería la inclusión de más **funciones integradas** para la manipulación de listas y conjuntos, así como para operaciones matemáticas avanzadas. Esto optimizaría el desarrollo de programas y reduciría la necesidad de implementar manualmente funcionalidades básicas. Además, la incorporación de **mecanismos para interactuar con bibliotecas externas** permitiría ampliar significativamente la funcionalidad de Aleph.

4.3 Conclusión final

A pesar de sus limitaciones, Aleph ofrece una confiabilidad y un conjunto de funcionalidades adecuadas para el desarrollo de programas y algoritmos basados en listas y conjuntos, como los utilizados en la teoría de autómatas. Con las mejoras necesarias, Aleph tiene el potencial de convertirse en una herramienta más potente y versátil, ideal para quienes deseen trabajar de manera eficiente con estas estructuras de datos.

Bibliografía:

- [1] R. W. Sebesta, *Concepts of Programming Languages*, 10th ed. Pearson, 2015.
- [2] K. C. Louden and K. A. Lambert, *Programming Languages: Principles and Practice*, 2nd ed. Cengage Learning, 2011.
- [3] J. R. Levine, *Flex & Bison: Text Processing Tools*. O'Reilly Media, 2009.